

In []:

```

nor=5
for j in range(nor):
    p=chr(65+j)
    val=int(input("Enter allocated resource of "+P))
    val

```

In [1]:

```

import pandas as pd
import numpy as np

```

In [7]:

```

x={'day':[1,2,3,4,5,6,7,8,9,10,11,12], 'city':['n','n','n','n','m','m','m','m','p','p','p','p']}
xd=pd.DataFrame(x)
xd.groupby('city').get_group('m')
print("mumbai temperature ",xd['temp'].max())

```

mumbai temperature 92

In [14]:

```

xd.loc[4:7,['temp']].mean()

```

Out[14]:

```

temp      88.5
dtype: float64

```

In [10]:

```

ar=[1,2,3,4]
print("Series:\n",pd.Series(ar,index=[4,5,3,'g']))
print("\nDataFrame:\n",pd.DataFrame(ar,index=[4,5,3,55],columns=[2]))

```

```

Series:
4      1
5      2
3      3
g      4
dtype: int64

```

```

DataFrame:
      2
4      1
5      2
3      3
55     4

```

In [23]:

```
di={'a':[1,2],9:'d'}
dl=di.keys()
print("Series:\n",pd.Series(dl,index=[4,5]))
print("DataFrame:\n",pd.DataFrame(di,index=[4,5]))
```

```
Series:
4      a
5      9
dtype: object
DataFrame:
   a  9
4  1  d
5  2  d
```

In [12]:

```
print(pd.Series(3,index=[5,4,22,11]))
pd.DataFrame(3,index=[1],columns=[1,2])
```

```
5      3
4      3
22     3
11     3
dtype: int64
```

Out[12]:

```
   1  2
1  3  3
```

In [33]:

```
import numpy as np
l=[1,2,3,4]
print(np.array(l,ndmin=2))
```

[[1 2 3 4]]

In [29]:

```
pd.Series(np.array([1,2,'a',4]),index=[345,3543,324,564])
```

Out[29]:

```
345      1
3543     2
324      a
564      4
dtype: object
```

In [36]:

```
data = {'Name':['Tom', 'nick', 'krish', 'jack'], 'Age':[20, 21, 19, 18]}
pd.DataFrame(data)
```

Out[36]:

	Name	Age
0	Tom	29
1	nick	29
2	krish	29
3	jack	29

In [40]:

```
data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'], 'Age':[27,np.nan, 22,np.nan], 'Address':[]}
df=pd.DataFrame(data,index=[5,6,7,8])
df
df[['Name', 'Age']]
```

Out[40]:

	Name	Age
5	Jai	27.0
6	Princi	NaN
7	Gaurav	22.0
8	Anuj	NaN

In [43]:

```
df['Age'].isnull()
```

Out[43]:

```
5    False
6     True
7    False
8     True
Name: Age, dtype: bool
```

In [47]:

```
df.notnull().sum()
```

Out[47]:

```
Name          4
Age            2
Address        4
Qualification  4
dtype: int64
```

In [48]:

```
df.isnull()
```

Out[48]:

	Name	Age	Address	Qualification
5	False	False	False	False
6	False	True	False	False
7	False	False	False	False
8	False	True	False	False

In [51]:

```
df
```

Out[51]:

	Name	Age	Address	Qualification
5	Jai	27	Delhi	Msc
6	Princi	24	Kanpur	MA
7	Gaurav	22	Allahabad	MCA
8	Anuj	32	Kannauj	Phd

In [14]:

```
df.to_csv("1.csv")
```

In [15]:

```
pd.read_csv('1.csv')
```

Out[15]:

	Unnamed: 0	Name	Age	Address	Qualification
0	5	Jai	27	Delhi	Msc
1	6	Princi	24	Kanpur	MA
2	7	Gaurav	22	Allahabad	MCA
3	8	Anuj	32	Kannauj	Phd

In [55]:

```
df=pd.read_csv('1.csv')
df
```

Out[55]:

	Unnamed: 0	Name	Age	Address	Qualification
0	5	Jai	27	Delhi	Msc
1	6	Princi	24	Kanpur	MA
2	7	Gaurav	22	Allahabad	MCA
3	8	Anuj	32	Kannauj	Phd

In [53]:

```
data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'], 'Age':[27, 24, 22, 32], 'Address':['Delhi', 'Kanpur', 'Allahabad', 'Kannauj'], 'Qualification':['Msc', 'MA', 'MCA', 'Phd']}
df=pd.DataFrame(data,index=[5,6,7,8])
print(df)
df.transpose()
```

	Name	Age	Address	Qualification
5	Jai	27	Delhi	Msc
6	Princi	24	Kanpur	MA
7	Gaurav	22	Allahabad	MCA
8	Anuj	32	Kannauj	Phd

Out[53]:

	5	6	7	8
Name	Jai	Princi	Gaurav	Anuj
Age	27	24	22	32
Address	Delhi	Kanpur	Allahabad	Kannauj
Qualification	Msc	MA	MCA	Phd

In [66]:

```
df.T
```

Out[66]:

	5	6	7	8
Name	Jai	Princi	Gaurav	Anuj
Age	27	24	22	32
Address	Delhi	Kanpur	Allahabad	Kannauj
Qualification	Msc	MA	MCA	Phd

In [21]:

```
df.transpose()
```

Out[21]:

	Age	27	24	22	32
Unnamed: 0		5	6	7	8
Name	Jai	Princi	Gaurav	Anuj	
Address	Delhi	Kanpur	Allahabad	Kannauj	
Qualification	Msc	MA	MCA	Phd	

In [55]:

```
print(df.ndim,df.shape,df.size)
```

2 (4, 4) 16

In [73]:

```
df
```

Out[73]:

	Name	Age	Address	Qualification
5	Jai	27	Delhi	Msc
6	Princi	24	Kanpur	MA
7	Gaurav	22	Allahabad	MCA
8	Anuj	32	Kannauj	Phd

In [59]:

```
df.loc[:,1,'Address':'Name']
```

Out[59]:

```
5
6
7
8
```

In [58]:

```
df.loc[5:7, 'Name': 'Address']
```

Out[58]:

	Name	Age	Address
5	Jai	27	Delhi
6	Princi	24	Kanpur
7	Gaurav	22	Allahabad

```
df.loc[5:7, 'Name': 'Address']
```

In [60]:

```
df.mean()
```

C:\Users\satish645\AppData\Local\Temp\ipykernel_6676\3698961737.py:1: Future Warning: Dropping of nuisance columns in DataFrame reductions (with 'numeric_only=None') is deprecated; in a future version this will raise TypeError. Select only valid columns before calling the reduction.

```
df.mean()
```

Out[60]:

```
Age    26.25
dtype: float64
```

In [61]:

```
df
```

Out[61]:

	Name	Age	Address	Qualification
5	Jai	27	Delhi	Msc
6	Princi	24	Kanpur	MA
7	Gaurav	22	Allahabad	MCA
8	Anuj	32	Kannauj	Phd

In [20]:

```
df[['Name', 'Qualification']]
```

Out[20]:

	Name	Qualification
5	Jai	Msc
6	Princi	MA
7	Gaurav	MCA
8	Anuj	Phd

In [22]:

```
df.at[7, 'Name']
```

Out[22]:

'Gaurav'

In [62]:

```
df
```

Out[62]:

	Name	Age	Address	Qualification
5	Jai	27	Delhi	Msc
6	Princi	24	Kanpur	MA
7	Gaurav	22	Allahabad	MCA
8	Anuj	32	Kannauj	Phd

In [65]:

```
df.iat[2,3]
```

Out[65]:

'MCA'

In [75]:

```
df.loc[7:8, "Name"]=["as", "fg"]  
df
```

Out[75]:

	Name	Age	Address	Qualification
5	Jai	27	Delhi	Msc
6	Princi	24	Kanpur	MA
7	as	as	as	MCA
8	fg	fg	fg	Phd

In [77]:

```
df.sort_values(by=['Name', 'Age'], ascending=False)
```

Out[77]:

	Name	Age	Address	Qualification
8	fg	fg	fg	Phd
7	as	as	as	MCA
6	Princi	24	Kanpur	MA
5	Jai	27	Delhi	Msc

In [78]:

df

Out[78]:

	Name	Age	Address	Qualification
5	Jai	27	Delhi	Msc
6	Princi	24	Kanpur	MA
7	as	as	as	MCA
8	fg	fg	fg	Phd

In [84]:

```
data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'], 'Age':[27,np.nan, 22,np.nan], 'Address':  
df=pd.DataFrame(data,index=[5,6,7,8])  
df
```

Out[84]:

	Name	Age	Address	Qualification
5	Jai	27.0	Delhi	Msc
6	Princi	NaN	Kanpur	MA
7	Gaurav	22.0	Allahabad	MCA
8	Anuj	NaN	Kannauj	Phd

In [85]:

```
df[(df['Age']>=24)&(df['Age']>=25)]
```

Out[85]:

	Name	Age	Address	Qualification
5	Jai	27.0	Delhi	Msc

In [86]:

```
df.describe()
```

Out[86]:

	Age
count	2.000000
mean	24.500000
std	3.535534
min	22.000000
25%	23.250000
50%	24.500000
75%	25.750000
max	27.000000

In [1]:

```
import numpy as np
```

In [2]:

```
pd.Series((1,2,3),index=[5,6,7])
```

```
-----  
NameError Traceback (most recent call last)  
Input In [2], in <cell line: 1>()  
----> 1 pd.Series((1,2,3),index=[5,6,7])  
  
NameError: name 'pd' is not defined
```

In [3]:

```
a=np.array((1,2,3),ndmin=2)  
a
```

Out[3]:

```
array([[1, 2, 3]])
```

In [4]:

```
a.ndim,a.size,a.shape
```

Out[4]:

```
(2, 3, (1, 3))
```

In [5]:

```
b=np.array([1,2,3,4],ndmin=8)  
b
```

Out[5]:

```
array([[[[[[[[1, 2, 3, 4]]]]]]]])
```

In [6]:

```
b.shape
```

Out[6]:

```
(1, 1, 1, 1, 1, 1, 1, 4)
```

In [7]:

```
b1=np.array([[1, 2, 3, 4],[1, 2, 3, 4],[1, 2, 3, 4]])  
b1.shape
```

Out[7]:

```
(1, 3, 4)
```

In [8]:

```
b1.size
```

Out[8]:

```
12
```

In [9]:

```
c=np.array([[1,2,3,7],[4,5,6,7]])  
c.shape
```

Out[9]:

```
(2, 4)
```

In [10]:

```
d=np.array([[1,2,3],[4,5,6]])  
d.shape  
d
```

Out[10]:

```
array([[1, 2, 3],  
       [4, 5, 6]])
```

In [11]:

```
d.shape=(6,1)  
d
```

Out[11]:

```
array([[1],  
       [2],  
       [3],  
       [4],  
       [5],  
       [6]])
```

In [12]:

```
d.shape=(1,6)  
d
```

Out[12]:

```
array([[1, 2, 3, 4, 5, 6]])
```

In [13]:

```
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5,  
6]]])  
arr  
arr.shape
```

Out[13]:

```
(2, 2, 3)
```

In [14]:

```
arr.shape=(2,6)  
arr
```

Out[14]:

```
array([[1, 2, 3, 4, 5, 6],  
       [1, 2, 3, 4, 5, 6]])
```

In [15]:

```
arr.size
```

Out[15]:

```
12
```

In [16]:

```
a=np.arange(2,24,3)  
a
```

Out[16]:

```
array([ 2,  5,  8, 11, 14, 17, 20, 23])
```

In [17]:

```
a.ndim
```

Out[17]:

```
1
```

In [27]:

```
b=a.reshape(2,4)
str(b).remove('\n')
```

```
-----
AttributeErrorTraceback (most recent call last)
Input In [27], in <cell line: 2>()
      1 b=a.reshape(2,4)
----> 2 str(b).remove('\n')

AttributeError: 'str' object has no attribute 'remove'
```

In [19]:

```
np.zeros(5,dtype='f')
```

Out[19]:

```
array([0., 0., 0., 0., 0.], dtype=float32)
```

In [20]:

```
np.zeros(5)
```

Out[20]:

```
array([0., 0., 0., 0., 0.])
```

In [132]:

```
a = np.arange(10)
b = a[2:7:2]
b
```

Out[132]:

```
array([2, 4, 6])
```

In [137]:

```
np.arange(1,10, dtype=float)
```

Out[137]:

```
array([1., 2., 3., 4., 5., 6., 7., 8., 9.])
```

In [139]:

```
np.arange(start=1, stop=10, step=1)
```

```
-----
TypeErrorTraceback (most recent call last)
Input In [139], in <cell line: 1>()
----> 1 np.arange(start='a', stop='z', step=1)

TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

In [31]:

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
a,b
```

Out[31]:

```
(array([[1, 2],
        [3, 4]]),
 array([[5, 6],
        [7, 8]]))
```

In [29]:

```
y = np.concatenate((a, b), axis=1)
z = np.concatenate((a, b), axis=0)
y,z
```

Out[29]:

```
(array([[1, 2, 5, 6],
        [3, 4, 7, 8]]),
 array([[1, 2],
        [3, 4],
        [5, 6],
        [7, 8]]))
```

In [35]:

```
arr = np.concatenate((a, b), axis=-1)
arr
```

Out[35]:

```
array([[1, 2, 5, 6],
       [3, 4, 7, 8]])
```

In [37]:

```
a = np.array([1,2,3,4],ndmin=5)
b = np.array([10,20,30,40],ndmin=10)
c = a * b
d = a+b
print(c)
print(d)
```

```
[[[[[[[ 10  40  90 160]]]]]]]]
[[[[[[[11 22 33 44]]]]]]]]
```

In [142]:

```
a=np.array([[0.0,0.0,0.0],[10.0,10.0,10.0],[20.0,20.0,20.0],[30.0,30.0,30.0]])
b = np.array([1.0,2.0,3.0])
print(a)
print(b)
```

```
[[ 0.  0.  0.]
 [10. 10. 10.]
 [20. 20. 20.]
 [30. 30. 30.]]
[1.  2.  3.]
```

In [16]:

```
print(a+b)
```

```
[[ 1.  2.  3.]
 [11. 12. 13.]
 [21. 22. 23.]
 [31. 32. 33.]]
```

In [17]:

```
sot= np.array([[3, 2, 4], [5, 0, 1]])
print(np.sort(sot))
```

```
[[2 3 4]
 [0 1 5]]
```

In [4]:

```
sorted([3,4])
```

Out[4]:

```
[3, 4]
```

In [18]:

```
sot.T
```

Out[18]:

```
array([[3, 5],
       [2, 0],
       [4, 1]])
```

In [19]:

```
sot.transpose()
```

Out[19]:

```
array([[3, 5],
       [2, 0],
       [4, 1]])
```

In [22]:

```
arr = np.array([[1, 2, 3, 4, 5, 6]])  
newarr = np.array_split(arr,2)  
print(newarr)
```

```
[array([[1, 2, 3, 4, 5, 6]]), array([], shape=(0, 6), dtype=int32)]
```

In [24]:

```
arr=np.array([41,42,52,43,567,4334])  
f=[]  
for ele in arr:  
    if ele>=50:  
        f.append(ele)  
    else:  
        continue  
print(f)
```

```
[52, 567, 4334]
```

In [145]:

```
a=np.array([1,2,3,4,5,6])  
b=[2,3,4]  
print(a[b])
```

```
[3 4 5]
```

In [27]:

```
from numpy import random  
x = random.randint(100, size=(3, 5))  
print(x)
```

```
[[78 61 83 77 35]  
 [63 55 68 95 31]  
 [77 27 48 86 56]]
```

In []:

```
y = random.choice([3, 5, 7, 9], size=(3, 5))  
print(y)
```

In [146]:

```

from urllib.request import urlopen
url="https://www.flipkart.com/"
page=urlopen(url)
html=page.read().decode("utf-8")
si=html.find("<title>")+len("<title>")
ei=html.find("</title>")
title=html[si:ei]

```

```

-----
-
gaierror                                Traceback (most recent call las
t)
File ~\anaconda3\lib\urllib\request.py:1346, in AbstractHTTPHandler.do_ope
n(self, http_class, req, **http_conn_args)
    1345 try:
-> 1346     h.request(req.get_method(), req.selector, req.data, headers,
    1347                 encode_chunked=req.has_header('Transfer-encoding'))
    1348 except OSError as err: # timeout error

File ~\anaconda3\lib\http\client.py:1285, in HTTPConnection.request(self,
method, url, body, headers, encode_chunked)
    1284 """Send a complete request to the server."""
-> 1285 self._send_request(method, url, body, headers, encode_chunked)

File ~\anaconda3\lib\http\client.py:1331, in HTTPConnection._send_request
(self, method, url, body, headers, encode_chunked)
    1330     body = _encode(body, 'body')
    1331     if self.debuglevel > 0:

```

In [5]:

```

import bs4
from bs4 import BeautifulSoup as bs
import requests

```

In [148]:

```

data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'], 'Age':[27, 22, 22, 32], 'A':[27.5, 24.5, 22.5, 32.5], 'Address':['Delhi', 'Kanpur', 'Allahabad', 'Kannauj'], 'Qualification':['Msc', 'MA', 'MCA', 'Phd']}
df=pd.DataFrame(data,index=[5,6,7,8])
df

```

Out[148]:

	Name	Age	A	Address	Qualification
5	Jai	27	27.5	Delhi	Msc
6	Princi	22	24.5	Kanpur	MA
7	Gaurav	22	22.5	Allahabad	MCA
8	Anuj	32	a	Kannauj	Phd

In [45]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 4 entries, 5 to 8
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Name            4 non-null      object
 1   Age             4 non-null      int64
 2   A               4 non-null      float64
 3   Address         4 non-null      object
 4   Qualification   4 non-null      object
dtypes: float64(1), int64(1), object(3)
memory usage: 192.0+ bytes
```

In [49]:

```
df.describe()
```

Out[49]:

	Age
count	4.000000
mean	26.250000
std	4.349329
min	22.000000
25%	23.500000
50%	25.500000
75%	28.250000
max	32.000000

In [149]:

```
df
```

Out[149]:

	Name	Age	A	Address	Qualification
5	Jai	27	27.5	Delhi	Msc
6	Princi	22	24.5	Kanpur	MA
7	Gaurav	22	22.5	Allahabad	MCA
8	Anuj	32	a	Kannauj	Phd

In [151]:

```
df.value_counts('Age')
```

Out[151]:

```
Age
22    2
27    1
32    1
dtype: int64
```

In [52]:

```
df.nunique()
```

Out[52]:

```
Name          4
Age           3
A             4
Address       4
Qualification  4
dtype: int64
```

In [55]:

```
df.nunique().sum()
```

Out[55]:

```
19
```

In [56]:

```
df.min()
```

```
C:\Users\satish645\AppData\Local\Temp\ipykernel_3540\3962516015.py:1: Future
Warning: Dropping of nuisance columns in DataFrame reductions (with 'numeric
_only=None') is deprecated; in a future version this will raise TypeError.
Select only valid columns before calling the reduction.
  df.min()
```

Out[56]:

```
Name          Anuj
Age           22
Address       Allahabad
Qualification  MA
dtype: object
```

In [32]:

df

Out[32]:

	Name	Age	Address	Qualification
5	Jai	27	Delhi	Msc
6	Princi	24	Kanpur	MA
7	Gaurav	22	Allahabad	MCA
8	Anuj	32	Kannauj	Phd

In [38]:

df.sum()

Out[38]:

```
Name          JaiPrinciGauravAnuj
Age              105
Address    DelhiKanpurAllahabadKannauj
Qualification      MscMAMCAPhd
dtype: object
```

In [59]:

df.aggregate([sum,min,max,'mean','median'])

C:\Users\satish645\AppData\Local\Temp\ipykernel_3540\4184609067.py:1: Future Warning: ['Name', 'A', 'Address', 'Qualification'] did not aggregate successfully. If any error is raised this will raise in a future version of pandas. Drop these columns/ops to avoid this warning.

```
df.aggregate([sum,min,max,'mean','median'])
```

Out[59]:

	Name	Age	Address	Qualification
sum	JaiPrinciGauravAnuj	103.00	DelhiKanpurAllahabadKannauj	MscMAMCAPhd
min	Anuj	22.00	Allahabad	MA
max	Princi	32.00	Kanpur	Phd
mean	NaN	25.75	NaN	NaN
median	NaN	24.50	NaN	NaN

In [52]:

df

Out[52]:

	Name	Age	Address	Qualification
5	Jai	27	Delhi	Msc
6	Princi	24	Kanpur	MA
7	Gaurav	22	Allahabad	MCA
8	Anuj	32	Kannauj	Phd

In [82]:

```
df.groupby('Age').first()
```

Out[82]:

	Name	Address	Qualification
Age			
22	Gaurav	Allahabad	MCA
24	Princi	Kanpur	MA
27	Jai	Delhi	Msc
32	Anuj	Kannauj	Phd

In [68]:

```
df.groupby('Address').get_group('Kanpur')
```

Out[68]:

	Name	Age	A	Address	Qualification
6	Princi	22	24.5	Kanpur	MA

In [14]:

```
import scipy.stats as sc
```

In [15]:

```
experience = [1,3,4,5,5,6,7,10, 11, 12, 15, 20, 25, 28, 30,35]
a=experience[::-1]
salary = [20000, 30000, 40000, 45000, 55000, 60000, 80000, 100000, 130000, 150000,
200000, 230000, 250000, 300000, 350000, 400000]
```

In [19]:

```
a,_=sc.pearsonr(experience,salary)
a
```

Out[19]:

```
0.9929845761480397
```

In [12]:

```
b,_=sc.spearmanr(experience,salary)
b
```

Out[12]:

```
0.9992644353546791
```

In [14]:

```
c,_=sc.kendalltau(experience,salary)
c
```

Out[14]:

0.9958246164193105

In [15]:

```
np.corrcoef(experience,salary)
```

Out[15]:

```
array([[1.          , 0.99298458],
       [0.99298458, 1.          ]])
```

In [91]:

```
print(a)
np.corrcoef(a,experience)
```

[35, 30, 28, 25, 20, 15, 12, 11, 10, 7, 6, 5, 5, 4, 3, 1]

Out[91]:

```
array([[ 1.          , -0.8330164],
       [-0.8330164,  1.          ]])
```

In [20]:

```
df=pd.DataFrame({'experience':[1,3,4,5,5,6,7,10, 11, 12, 15, 20, 25, 28, 30,35],  
                 'salary':[20000, 30000, 40000, 45000, 55000, 60000, 80000, 100000, 130000,  
200000, 230000, 250000, 300000, 350000, 400000]})  
df
```

Out[20]:

	experience	salary
0	1	20000
1	3	30000
2	4	40000
3	5	45000
4	5	55000
5	6	60000
6	7	80000
7	10	100000
8	11	130000
9	12	150000
10	15	200000
11	20	230000
12	25	250000
13	28	300000
14	30	350000
15	35	400000

In [26]:

```
df['experience'].corr(df['salary'])
```

Out[26]:

0.9929845761480398

In [28]:

```
df['salary'].corr(df['experience'],method="spearman")
```

Out[28]:

0.9992644353546792

In [29]:

```
df['salary'].corr(df['experience'],method="kendall")
```

Out[29]:

0.9958246164193105

In [30]:

```
df['salary'].corr(df['experience'],method="pearson")
```

Out[30]:

0.9929845761480398

In [31]:

```
df.corr()
```

Out[31]:

	experience	salary
experience	1.000000	0.992985
salary	0.992985	1.000000

In [33]:

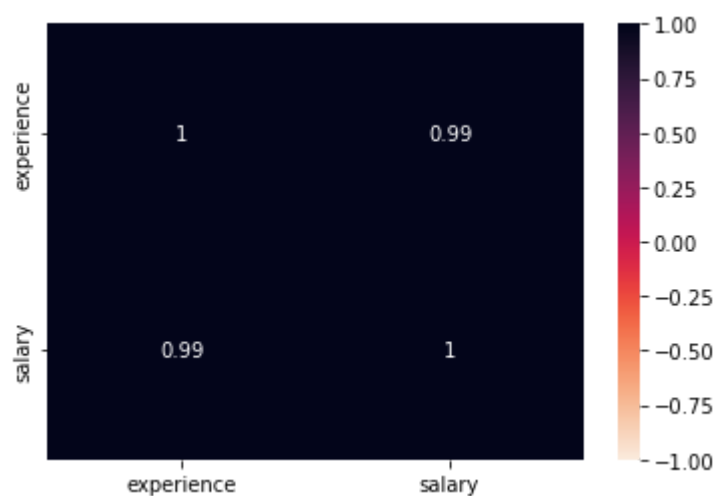
```
import seaborn as sb
```

In [42]:

```
sb.heatmap(df.corr(),vmax=1,vmin=-1,annot=True,cmap="rocket_r")
```

Out[42]:

<AxesSubplot:>



In [44]:

```
from sklearn.linear_model import LinearRegression
```

In [63]:

```
model = LinearRegression()
```

In [66]:

```
x=np.array(experience)
y=np.array(salary)
```

In [38]:

```
from urllib.request import urlopen
```

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: